NTNU

Data-Driven Modeling

From linear regression to deep networks — and where pure data falls short

Adil Rasheed · Introduction
NTNU — Department of Engineering Cybernetics

Outline

- Motivation: learning models from data
- From linear regression to deep networks
- Cost functions and optimization
- Generalization: train, validation, test
- Regularization
- Uncertainty quantification
- PDE benchmark I: a heat-field surrogate
- PDE benchmark II: a flow-reconstruction surrogate
- ODE benchmark: a black-box vessel model
- ODE benchmark II: a wind-turbine surrogate
- Synthesis and outlook

01

Motivation: learning models from data

Data-driven modeling

Instead of deriving equations from first principles, **fit a model to data**. Given examples $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$, learn a map $\hat{y} = f_{\boldsymbol{\theta}}(\boldsymbol{x})$ by tuning parameters $\boldsymbol{\theta}$.

Ingredients

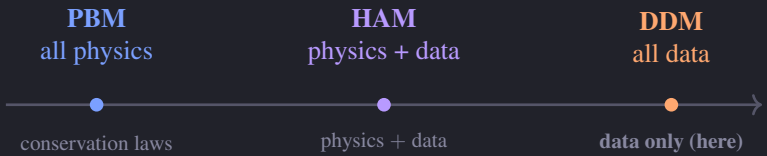
- **Data**: inputs $\boldsymbol{x}$, targets y
- **Model** $f_{\boldsymbol{\theta}}$ (regression $\rightarrow$ deep net)
- **Cost** measuring misfit
- **Optimizer** tuning $\boldsymbol{\theta}$

When it shines

- Equations unknown or too costly
- Plenty of representative data
- Fast repeated inference needed

This lecture: **supervised regression** — the workhorse of scientific ML.

The modeling spectrum, revisited



DDM strengths

Flexible, fast at inference, captures effects no one has modeled.

DDM weaknesses

Data-hungry; poor extrapolation; no conservation laws; can fail silently.

We sit at the far-right of the spectrum today — and feel its limits.

Supervised learning in one picture

1. Collect data $\{(\boldsymbol{x}_i, y_i)\}$ and **split** it (train / validation / test).
2. Choose a model family $f_{\boldsymbol{\theta}}$ and a cost $\mathscr{L}(\boldsymbol{\theta})$.
3. **Train**: minimize the cost on the training set.
4. **Select** (validation) and **report** (test) honestly.
5. **Quantify uncertainty** of the predictions.

Every data-driven method in this course is some version of these five steps — what changes is the model, the cost, or how honestly the last two are done.

02

From linear regression to deep networks

Linear regression: the simplest model

Fit a line/hyperplane $\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$ by minimizing squared error. Stacking data into $\mathbf{X}$ (rows = samples) and $\boldsymbol{y}$:

$$\min_{\boldsymbol{w},b} \left\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\right\|^2 \quad \Rightarrow \quad \boldsymbol{w}^\star = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{y} \ \text{(normal equations)}.$$

Why start here

- Closed-form solution, convex
- Interpretable coefficients
- The template for everything that follows

Limitation

Only linear relationships — real data is rarely linear.

Basis functions: linear in parameters, nonlinear in inputs

Keep the easy linear-least-squares machinery but enrich the inputs with fixed features $\boldsymbol{\phi}(\boldsymbol{x})$:

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}), \qquad \boldsymbol{\phi} = \left[1, x, x^2, \dots\right] \text{ (polynomial), or RBFs, Fourier.}$$

Polynomial regression

Captures curvature; still solved by least squares.

The catch

You must choose the features. High-degree bases overfit; good features are hard to hand-craft in high dimensions.

Key idea: let the model *learn* its own features $\rightarrow$ neural networks.

Neural networks: learned features

A multilayer perceptron (MLP) stacks affine maps and nonlinear activations:

$$\boldsymbol{h}^{(\ell)} = \sigma\big(\mathbf{W}^{(\ell)}\boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}\big), \qquad \hat{y} = \mathbf{W}^{(L)}\boldsymbol{h}^{(L-1)} + b^{(L)}.$$

Why it works

- **Universal approximation**: enough width/depth fits any continuous function.
- Hidden layers *learn* the basis $\boldsymbol{\phi}$.
- Activations σ (ReLU, tanh, GELU) supply nonlinearity.

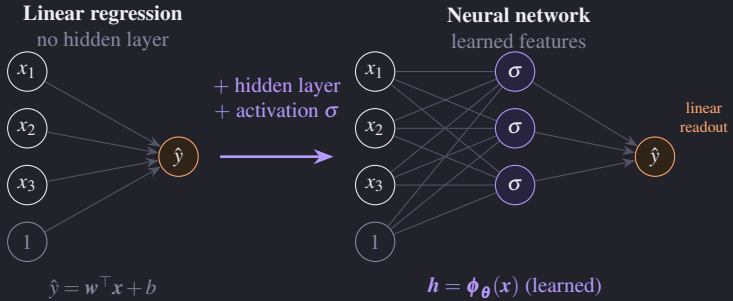
The price

Non-convex; no closed form; needs iterative optimization (next section) and care against overfitting.

Parameters $\boldsymbol{\theta} = \{\mathbf{W}^{(\ell)}, \boldsymbol{b}^{(\ell)}\}$; "deep" = many layers.

From a linear unit to a network

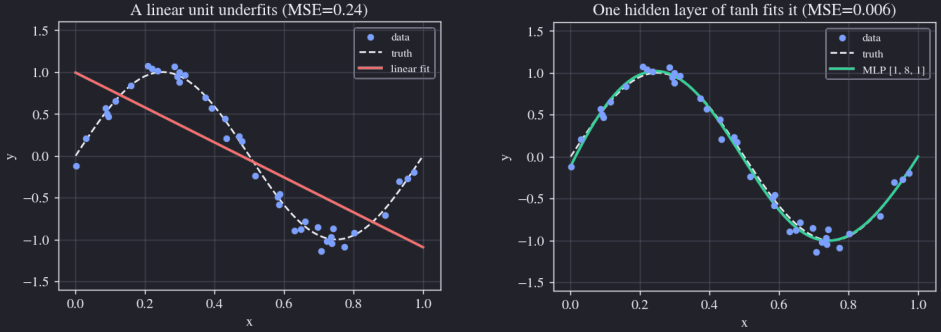
A neural network is linear regression with *extra layers*: the same inputs, the same linear output — but a **learned** feature layer in between.



Right: $\boldsymbol{h} = \sigma(\mathbf{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$, then the *same* linear readout $\hat{y} = \mathbf{W}^{(2)}\boldsymbol{h} + b^{(2)}$. Linear regression is just the case with **no** hidden layer.

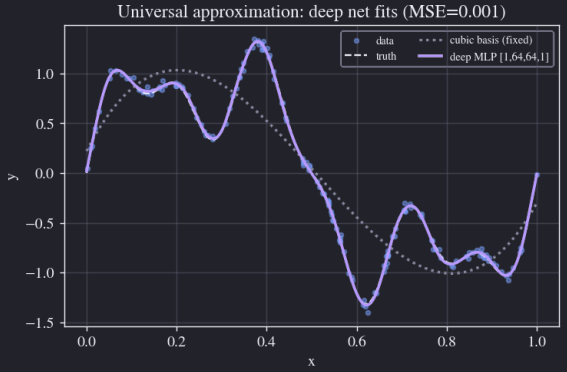
A line is not enough

Fit the curved target $y = \sin(2\pi x)$ from noisy samples. A *linear unit* (no hidden layer) can only draw a straight line; adding *one hidden layer of* `tanh` *units* lets the model bend to the data — same cost, same optimizer.



Setup: 40 noisy points, noise $\sigma = 0.08$; left is the closed-form least-squares line (no hidden layer), right an MLP $[1, 8, 1]$ of `tanh` units. The MSE drops from 0.24 to 0.006: the hidden layer supplies the learned features $\boldsymbol{\phi}_{\boldsymbol{\theta}}(x)$ that the line lacked.

Deep nets are universal approximators



Universal approximation

A network with enough width/depth can approximate *any* continuous function to arbitrary accuracy.

No features hand-picked

A fixed cubic basis (dotted) misses the wiggles; the deep `tanh` net learns its own basis and tracks them (MSE = 0.001 vs 0.13).

Setup: target $\sin 2\pi x + \frac{1}{2}\sin 6\pi x + \frac{1}{4}\sin 12\pi x$, 160 pts ($\sigma$=0.03); deep MLP $[1, 64, 64, 1]$ vs a fixed cubic basis.

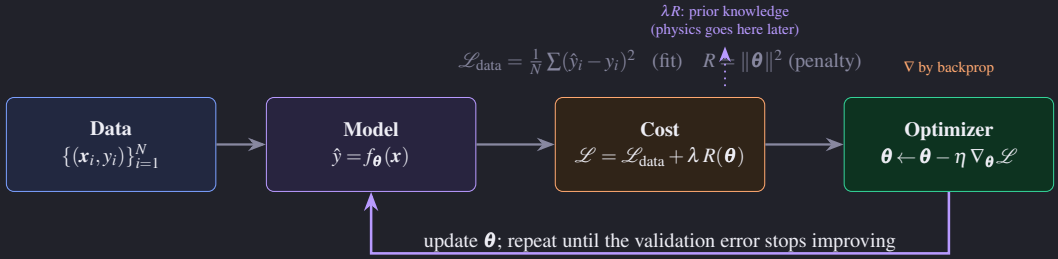
This flexibility is exactly why deep nets dominate — and, later, exactly why they need regularization and honest evaluation.

03

Cost functions and optimization

Anatomy of a data-driven model

Four ingredients, wired into one loop — *data*, a *model*, a *cost* (fit + penalty), an *optimizer*. Later methods change just **one** box.



The rest of Part I details each box: cost (here), optimizer (next), train/val/test, regularization, uncertainty.

Cost functions: what “good fit” means

The cost (loss) $\mathscr{L}(\boldsymbol{\theta})$ scores predictions against targets. Choose it to match the task:

Regression

$$\text{MSE} = \frac{1}{N}\sum_i (\hat{y}_i - y_i)^2$$

- MSE: smooth, penalizes large errors (Gaussian noise).
- $\text{MAE} = \frac{1}{N}\sum_i |\hat{y}_i - y_i|$: robust to outliers.

Classification

$$\text{CE} = -\frac{1}{N}\sum_i \sum_c y_{ic} \log \hat{p}_{ic}$$

Cross-entropy on softmax probabilities.

Probabilistic view

MSE $\equiv$ maximum likelihood under Gaussian noise; cross-entropy $\equiv$ MLE for a categorical model. The loss encodes your noise assumption.

Gradient descent and its stochastic cousins

Minimize $\mathscr{L}$ by stepping downhill: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathscr{L}$.

Variants

- **Batch** GD: full dataset per step (exact, slow).
- **SGD**: one mini-batch per step (noisy, fast, escapes shallow minima).
- **Adam**: per-parameter adaptive step from gradient moments — the default.

Learning rate η

The single most important knob. Too large $\rightarrow$ diverge; too small $\rightarrow$ crawl. Use schedules / decay.

Backpropagation computes $\nabla_{\boldsymbol{\theta}} \mathscr{L}$ exactly and cheaply by the chain rule (reverse-mode autodiff).

Anatomy of a training run

1. Initialize $\boldsymbol{\theta}$ (Glorot/He); standardize inputs/outputs.
2. For each epoch, for each mini-batch: forward pass $\rightarrow$ loss $\rightarrow$ backprop $\rightarrow$ optimizer step.
3. Track **training and validation loss** every epoch.
4. Stop when validation loss stops improving (next section).

Practical defaults

Adam, learning rate 10^{-3} with decay, mini-batch 32–256, smooth activations, standardized data. Monitor the *loss curves*, not just the final number.

04

Generalization: train, validation, test

The only question that matters: does it generalize?

A model is useful only if it works on *unseen* data. Split the dataset:

Train

Fit $\boldsymbol{\theta}$ (minimize the cost).

Validation

Tune hyperparameters; choose when to stop.

Test

Touch *once*, at the end, to report honest performance.

Golden rule

The test set must never influence training or model choice — otherwise you are reporting fitted, not predicted, performance.

Underfitting, overfitting, and the sweet spot



- **Underfit**: model too simple — high train *and* validation error (high bias).
- **Overfit**: model memorizes noise — low train, high validation error (high variance).
- **Bias–variance trade-off**: total error balances the two.

Diagnose by the *gap* between training and validation curves.

Tools for honest generalization

Early stopping

Halt when validation loss stops improving — the cheapest, most effective regularizer.

Model selection

Compare architectures / hyperparameters on validation; report the chosen one on test.

Cross-validation

k-fold: rotate the validation split to use scarce data efficiently and estimate variance.

Distribution shift

Even a great test score fails if deployment data differs — the seed of DDM's extrapolation problem (Part II).

05

Regularization

Regularization: fighting overfitting

Add a penalty (or a constraint) that discourages needless complexity, trading a little training fit for much better generalization.

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \underbrace{\mathcal{L}_{\text{data}}(\boldsymbol{\theta})}_{\text{fit}} + \lambda \underbrace{R(\boldsymbol{\theta})}_{\text{penalty}}.$$

Weight penalties

- **L2** (ridge): $R = \|\boldsymbol{\theta}\|_2^2$ — shrinks weights smoothly (= weight decay).
- **L1** (lasso): $R = \|\boldsymbol{\theta}\|_1$ — drives weights to *exactly zero* (sparsity, feature selection).

Strength λ

A hyperparameter chosen on the *validation* set: larger λ = simpler model.

Regularizers for neural networks

Dropout

Randomly zero a fraction of units each step → prevents co-adaptation, acts like an ensemble. (Reused for uncertainty next.)

Data augmentation

Synthesize plausible new samples (noise, transforms) → more effective data, better invariances.

Early stopping

Stop at best validation loss — implicit regularization (cf. previous section).

Normalization

Batch/layer norm and input standardization stabilize and speed training.

Physics as the ultimate regularizer

Physics-informed and hybrid models add a *physics* penalty / correction — the most informative regularizer of all, and why they need far less data.

06

Uncertainty quantification

Why a point prediction is not enough

A model that only outputs $\hat{y}$ cannot say "I don't know" — dangerous for decisions. We want a predictive *distribution*, not a number.

Aleatoric uncertainty

Irreducible *noise in the data* (sensor noise, randomness). Stays even with infinite data. Model it by predicting a variance.

Epistemic uncertainty

Model uncertainty from limited data — large where data is sparse or out-of-distribution. Reducible with more data.

The crucial one for us

Epistemic uncertainty *should* blow up under extrapolation — a built-in warning that the DDM is guessing (Part II).

Practical UQ methods

Deep ensembles

Train M networks from different seeds; prediction = mean, uncertainty = spread. Simple, strong, embarrassingly parallel.

Heteroscedastic head

Network outputs mean *and* variance $(\hat{\mu}, \hat{\sigma}^2)$; train by Gaussian negative log-likelihood (aleatoric).

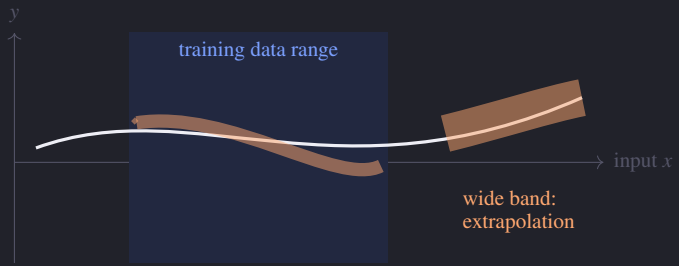
MC dropout

Keep dropout *on at test time*; average many stochastic forward passes. Cheap approximate Bayesian inference.

Calibration

A 90% interval should contain truth 90% of the time — check on a held-out set.

Uncertainty in one picture



Epistemic uncertainty is narrow where data is dense, wide where the model extrapolates — the model flags its own ignorance.

07

PDE benchmark I: a heat-field surrogate

What we are solving

Learn the temperature field of the Chapter-1 heat benchmark *directly from solver data*, with no governing equation. Train an MLP

$$\hat{T}_{\boldsymbol{\theta}}(x, y, \alpha) \;\approx\; T(x, y, t^{*}; \alpha),$$

where α is the thermal diffusivity — the uncertain material property.

The generalization test

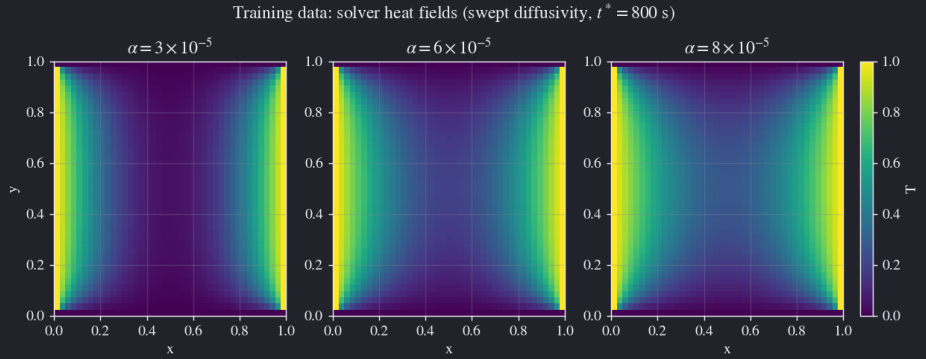
- **Train:** a band of diffusivities $\alpha \in [3, 8] \times 10^{-5}$.
- **Interpolate:** held-out α *inside* the band.
- **Extrapolate:** α *outside* the band.

Why a surrogate at all

Once trained, a prediction is a single forward pass — orders of magnitude cheaper than re-running the finite-difference solver for each α.

Notebook: `ddm_heat_surrogate.ipynb` (uses the heat benchmark solver).

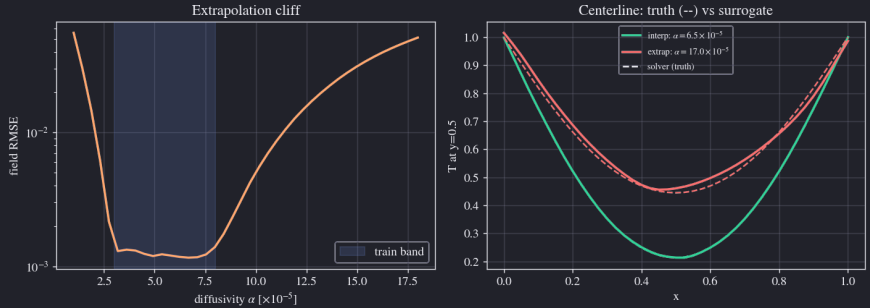
How the data is generated



Setup: FTCS solver on a 41×41 grid; Dirichlet T=1 (left/right), T=0 (top/bottom), initial T=0; field sampled at t^*=800 s; six runs over the training band $\alpha \in \{3, \ldots, 8\} \times 10^{-5}$.

Run the Chapter-1 finite-difference solver at each training diffusivity and sample the field at a fixed time t^*. Higher α diffuses the hot left/right edges further into the cold interior. The surrogate's job is to learn this parametric family — and to keep working for diffusivities it never saw (`ddm_heat_surrogate.ipynb`).

Interpolation is easy; extrapolation is a cliff



Left: field RMSE is tiny inside the training band, $\sim 40\times$ larger outside. *Right:* the interpolated centerline (green) sits on the solver truth; the extrapolated one (red) departs.

Verdict $\rightarrow$ the case for HAM

Accurate *inside* its training band, wrong *outside*, with no conservation to fall back on. Keep the heat equation and learn only the uncertain part — accuracy survives.

08

PDE benchmark II: a flow-reconstruction surrogate

What we are solving

Reconstruct the velocity field of the Chapter-1 **lid-driven cavity** from sparse measurements, with no momentum equation. Train an MLP

$$(x, y) \longmapsto (\hat{u}, \hat{v})$$

on velocity samples — but only from the *lower part* of the cavity, as if the sensors could not reach the moving lid.

The generalization test (in space)

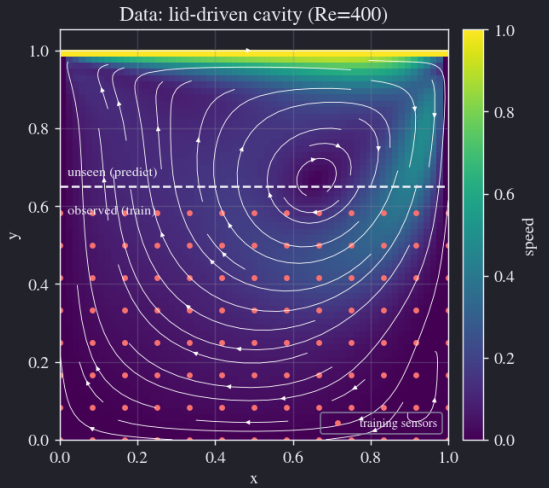
- **Train:** samples with $y \leq 0.65$.
- **Interpolate:** held-out points *inside* that region.
- **Extrapolate:** the unobserved band $y > 0.65$ under the lid.

Why this matters

A digital twin rarely has sensors everywhere. Can a black box fill in the region it never measured?

Notebook: `ddm_ns_surrogate.ipynb` (uses the cavity Navier–Stokes solver).

How the data is generated

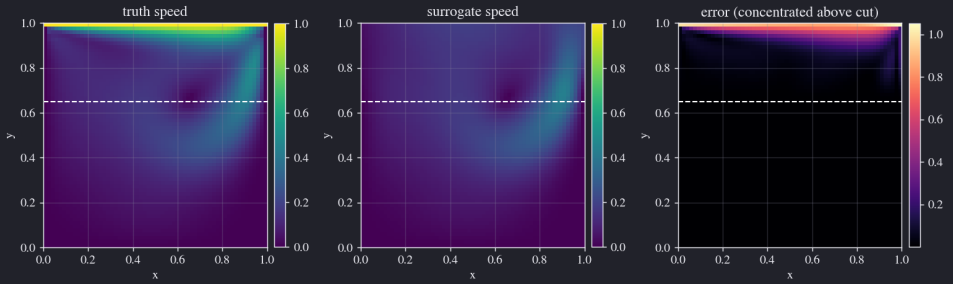


- One projection-method solve of the cavity at $\mathrm{Re} = 400$ on a 61×61 grid.
- "Sensors" sample the velocity only *below* the dashed line.
- The lid drives a thin, fast shear layer along the very top — entirely in the unobserved band.

The hidden feature

Nothing in the training data hints that $|\boldsymbol{u}|$ jumps to the lid speed at the top.

Interpolation is easy; extrapolation is a cliff



Below the cut the reconstruction is near-exact (interpolation RMSE $\approx 5 \times 10^{-4}$); above it the surrogate misses the lid boundary layer entirely and the error is *hundreds of times* larger — with no warning.

Verdict → the case for HAM

A black box fills unobserved regions confidently and wrongly. A physics-informed model enforces the Navier–Stokes residual *everywhere*, recovering the unmeasured boundary layer.

09

ODE benchmark: a black-box vessel model

Learning vessel dynamics from data

Skip the equations of motion; learn the one-step map from data,

$$\boldsymbol{\nu}_{n+1} = f_{\boldsymbol{\theta}}(\boldsymbol{\nu}_n, \boldsymbol{\tau}_n) \quad \big(\text{or } \dot{\boldsymbol{\nu}} = f_{\boldsymbol{\theta}}(\boldsymbol{\nu}, \boldsymbol{\tau})\big),$$

trained on logged maneuvers, then *rolled out* autoregressively for prediction.

What works

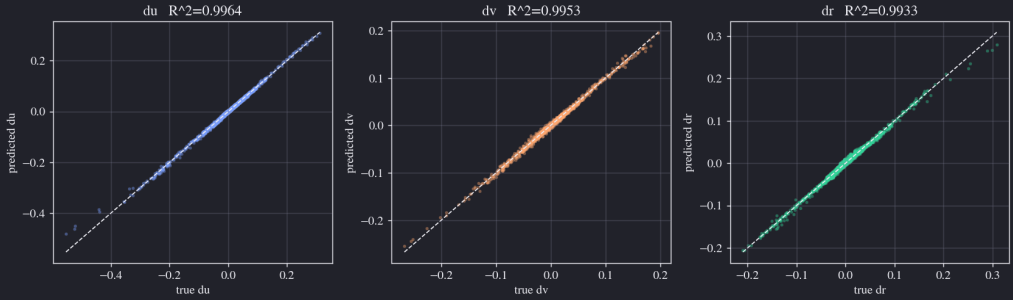
Fits the training maneuvers well; no hydrodynamic coefficients needed.

What breaks

- Errors **compound** over the rollout (drift).
- New maneuvers / speeds → off-distribution failure.
- May violate momentum, energy, symmetry.

Notebook: `ddm_vessel_surrogate.ipynb` (uses the vessel benchmark).

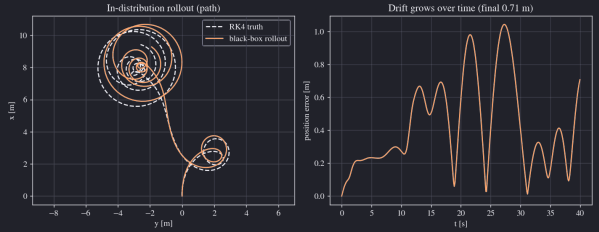
One step looks almost perfect



Setup: one-step map trained on 40 smooth Fossen 3-DOF maneuvers (Δt=0.1 s, T=40 s horizon; $\sim$16,000 one-step pairs).

Held-out one-step predictions of the velocity increment $\Delta\boldsymbol{\nu}$ against truth: the scatter sits on the diagonal, R^2 all but unity. This is the seductive part — and it says little about behaviour under rollout (`ddm_vessel_surrogate.ipynb`).

Drift: small step errors compound

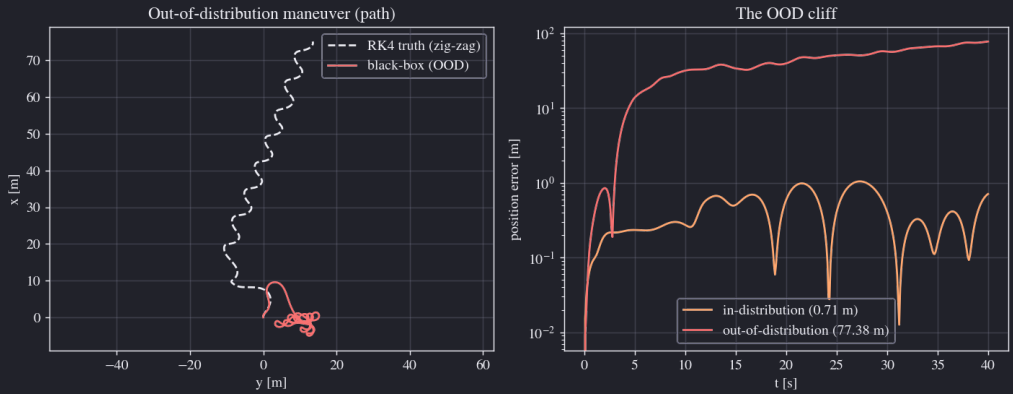


- One-step error is tiny — but rollout feeds outputs back as inputs, so error **accumulates**.
- No physical constraint anchors long horizons.
- Mitigations help; the structural limit remains.

This motivates hybrids

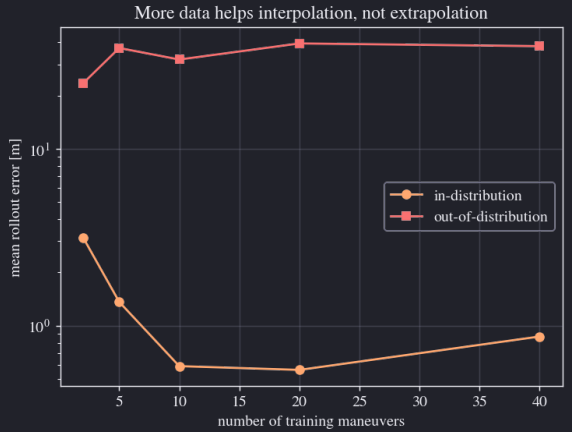
Keep the physics, learn only the residual (CoSTA) — drift disappears.

The out-of-distribution cliff



Trained on smooth maneuvers, the surrogate is asked to roll out a *zig-zag* (sharp, heading-triggered reversals). It collapses into a knot while the truth sails north — the rollout error is an order of magnitude larger than in-distribution (note the log axis). Interpolation is easy; extrapolation is a cliff.

Does more data help?

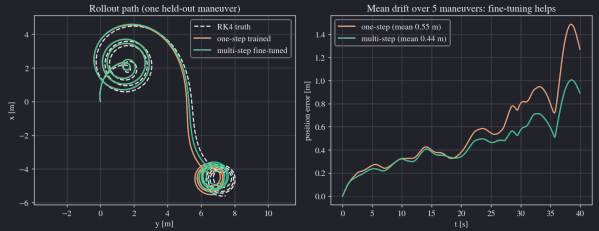


- More training maneuvers steadily improve the **in-distribution** rollout — better interpolation.
- The **out-of-distribution** error barely moves.
- No amount of smooth-maneuver data teaches sharp reversals.

The lesson

Data fixes *interpolation*, not *extrapolation*.

A partial fix: fine-tune on rollouts



- Train/test mismatch: trained on *one* step, used for *many*.
- Fine-tuning on a k=12-step rollout targets the right objective.
- Mean drift drops — but the OOD cliff is untouched.

Patch, not cure

It softens drift; only physics removes the structural failure.

10

ODE benchmark II: a wind-turbine surrogate

What we are solving

Learn the steady **power curve** of the Chapter-1 wind turbine from ODE data, with no controller model. Train an MLP

$$\hat{P}_{\boldsymbol{\theta}}(v) \;\approx\; P(v),$$

mapping wind speed to generated power. The controller has *two regimes* the surrogate must respect.

The generalization test

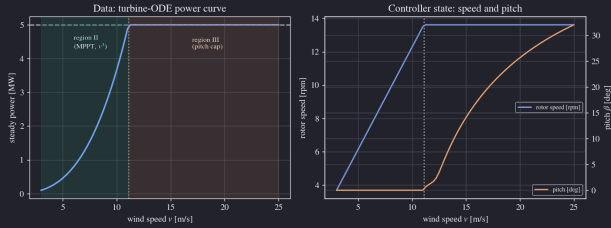
- **Train:** region II, $v < v_{\text{rated}}$ (power rises as v^3).
- **Interpolate:** held-out v *inside* region II.
- **Extrapolate:** region III, $v \geq v_{\text{rated}}$ (pitch-capped).

The regime change

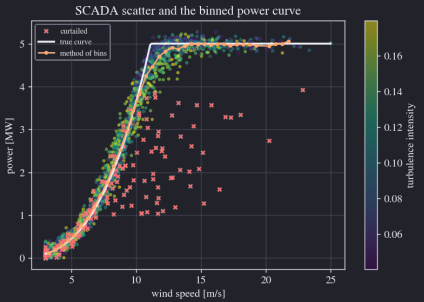
Above rated wind a pitch controller *caps* power at 5 MW to protect the drivetrain — an action absent from the region-II data.

Notebook: `ddm_wind_ode_surrogate.ipynb` (uses the turbine benchmark).

How the data is generated



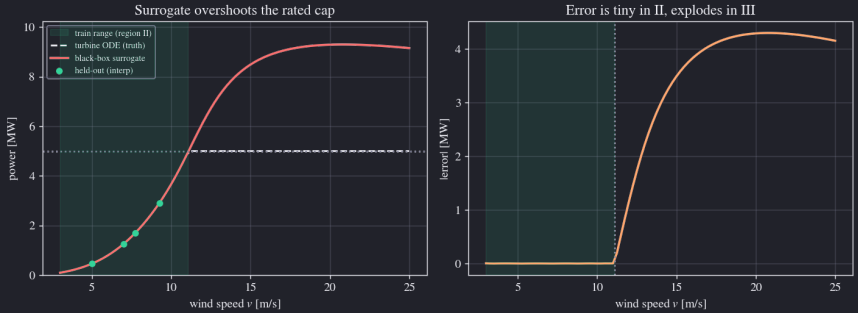
From the turbine ODE: MPPT below rated, pitch cap above.



Real turbines: the same curve, from noisy 10-min SCADA averages (`ddm_wind_power_curve.ipynb`).

Setup: NREL-5MW-like rotor. Sweep the wind speed ($v = 3 \to 25$ m/s, 89 points; v_{rated}=11.1 m/s, rated cap 5 MW) and integrate the rotor ODE to steady state at each v; the controller traces out region II (MPPT) then region III (pitch cap).

Interpolation is easy; extrapolation is a cliff



Trained only on region II, the surrogate matches the held-out points there exactly, then *extrapolates the cubic rise past the rated cap* — predicting ~ 9 MW where the machine holds 5 MW. The error explodes in region III.

Verdict → the case for HAM

The failure is a missing *mechanism* (the pitch controller), not missing data. Keep the controller, learn only the uncertain aerodynamics — valid across both regimes.

11

Synthesis and outlook

Data-driven modeling: the balance sheet

Strengths

- Needs no governing equations
- Captures effects no one has modeled
- Fast inference once trained
- Flexible across domains

Limits

- Data-hungry
- Poor extrapolation (the cliff, the drift)
- No conservation / no guarantees
- Can fail silently — hence UQ

The take-home

DDM is indispensable *and* insufficient on its own for safety-critical, extrapolative scientific modeling — which is why the course does not stop here.

How the rest of the course fixes these limits

Limit of DDM	The hybrid fix
No physics in the loss	physics-informed networks (residual in loss)
Black box, no operator	deep operator learning
Cost of many solver runs	Fourier neural operator
Extrapolation / drift	corrective source terms (physics + correction)
No interpretable model	data-driven equation discovery
Too many dimensions	reduced-order modeling

Foundation laid

Every method above reuses today's building blocks — networks, losses, optimizers, train/val/test, regularization, UQ — now *constrained by physics*.

Summary

1. DDM fits models to data: **data** $\rightarrow$ **model** $\rightarrow$ **cost** $\rightarrow$ **optimizer**.
2. Regression generalizes to deep nets via *learned* features (universal approximation).
3. Honest evaluation needs **train / validation / test**; diagnose bias vs variance.
4. **Regularization** (L1/L2, dropout, early stopping) curbs overfitting.
5. **Uncertainty quantification** tells you when not to trust a prediction.
6. On the benchmarks, pure DDM interpolates well but **extrapolates badly** — motivating physics-informed and hybrid methods.



NTNU

Data is powerful — physics makes it trustworthy

Adil Rasheed · NTNU · adil.rasheed@ntnu.no